Python-Pydantic-M-Set1

FDE / Architect practice | Reasoning, alternatives, and design takeaways

01. A default argument outlives a request

Answer B: ['A', 'B']; default to None and create a list inside the function.

The default list is created when the function is defined. The first call mutates it to [A]; the second appends B to that same object. Returning a copy protects the returned value from later mutations, but it does not reset the shared default. A None sentinel allows the function to allocate a new list for each omitted argument.

Why the alternatives fail

A. Function calls reuse the same default object; a list literal in the signature is not evaluated per call.

C. Copying on return leaves the default list contaminated for the next request.

D. extend([doc_id]) mutates the same shared list just as append does here.

Design takeaway: Decide whether each mutable object belongs to one call, one instance, or the whole process.

Official references: [1]

02. A shallow copy shares nested objects

Answer C: ['base', 'urgent']; assign request['tags'] = template['tags'].copy() before append.

The two dictionaries are different objects, but their tags values initially point to one list. Mutating that list is visible through both dictionaries. Copying the tags list breaks the only nested alias in this example. A deeper object graph would require an ownership-aware copy strategy, potentially deepcopy.

Why the alternatives fail

A. dict(template) also makes a shallow copy and preserves the shared list reference.

B. A shallow dictionary copy duplicates the mapping, not the objects stored as its values.

D. List += mutates the list in place here; it does not isolate the template.

Design takeaway: A new container does not imply new nested state.

Official references: [2]

03. A cache fallback runs eagerly

Answer A: fetch then 7; use cache[key] if key in cache else fetch().

Python evaluates call arguments before calling dict.get. fetch() therefore runs even though its result is unused. A conditional expression evaluates only the selected branch. Checking key membership also distinguishes an absent key from a present key whose value is None, zero, or another false-like value.

Why the alternatives fail

B. dict.get receives an already evaluated default value, not a lazy callback.

C. The or expression calls fetch for a cached None, violating the stated requirement.

D. Evaluating the default neither overwrites the dictionary nor changes the value selected by get.

Design takeaway: Inspect evaluation order when a seemingly cheap expression includes I/O.

Official references: [3]

04. Validation consumes an iterator

Answer D: An empty list; materialize once, then validate and persist that list.

The generator is single-pass. Because every check succeeds, all() consumes it through exhaustion. list(rows) then has nothing left. Materializing a small batch first provides a reusable sequence. For a large stream, design validation and persistence as an explicit streaming process instead of silently relying on replay.

Why the alternatives fail

A. iter() on an iterator returns that iterator, not a restarted data source.

B. all() obtains items from its input iterator as it checks them.

C. The final item has also been consumed; all() does not put it back.

Design takeaway: Know whether a boundary accepts a reusable collection or a one-shot stream.

Official references: [4]

05. Loop callbacks capture a binding

Answer A: ['C', 'C', 'C']; use lambda name=name: name.

Each callback looks up the same name binding when it is called. By then the loop has left name equal to C. A default argument is evaluated when each lambda is created, capturing that iteration's string. A small factory function with its own parameter would also work.

Why the alternatives fail

B. Python closures do not automatically snapshot each loop value.

C. Reversing iteration only changes the final shared value; it does not create independent bindings.

D. Renaming the shared variable leaves the binding behavior unchanged.

Design takeaway: Registration time and execution time can observe different values.

Official references: [5]

06. Pairing must reject missing results

Answer C: pairs = list(zip(ids, vectors, strict=True))

zip with strict=True raises ValueError when iteration reveals unequal lengths. Wrapping it in list forces that check before execution reaches write(). Merely constructing a strict zip object does not check the lengths yet; a writer consuming it incrementally could write earlier pairs before the mismatch appears.

Why the alternatives fail

A. The lazy iterator may fail inside write(), after some writes have already occurred.

B. Ordinary zip truncates to the shorter input, silently losing the third ID.

D. Truncating IDs hides the missing embedding instead of rejecting the incomplete batch.

Design takeaway: Validation timing matters when a downstream consumer has side effects.

Official references: [6]

07. Static annotations are not an ingress validator

Answer D: KeyError; validate the payload before accessing the required field.

TypedDict describes the expected dictionary shape to static tooling. It does not intercept this assignment, add fields, or validate JSON at runtime. The actual dictionary is empty, so subscription raises KeyError. A runtime validator can reject the malformed request with a useful structured error before business logic runs.

Why the alternatives fail

A. TypedDict does not populate default values.

B. An annotation does not execute a runtime shape check.

C. Dictionary subscription raises for a missing key; only an explicit get could return a default.

Design takeaway: Use static typing inside trusted code and runtime validation at untrusted boundaries.

Official references: [7]

08. Cleanup can erase an exception

Answer B: The string 'done'; remove return from finally and perform cleanup there.

A return executed in finally determines the function result and suppresses the pending exception. Since send raises, the return of ok is never reached. Put cleanup in finally without returning from it; the original ConnectionError will then propagate unless explicitly handled elsewhere.

Why the alternatives fail

A. Control-flow statements in finally can replace an exception that was propagating.

C. The exception interrupts execution before return ok.

D. Returning from an except block handles the failure and returns a value instead of propagating it.

Design takeaway: Cleanup should release resources without rewriting the operation's outcome.

Official references: [8]

09. Translate an exception without losing its cause

Answer D: except TimeoutError as exc: raise RetrievalUnavailable("search failed") from exc

raise NewError(...) from exc establishes an explicit cause while changing the exception type seen by callers. This creates a stable service-facing error contract without discarding the underlying failure. Logs can retain the chain; a public response can expose only the permitted message.

Why the alternatives fail

A. Reraising exc preserves the lower-level type rather than translating it to RetrievalUnavailable.

B. from None suppresses display of the implicit context and does not set the requested explicit cause.

C. Returning an exception object is an ordinary successful return, not an exception being raised.

Design takeaway: Separate the public error contract from the diagnostic evidence.

Official references: [9]

10. A lazy reader outlives its file

Answer B: Iteration uses a closed file; yield stripped lines inside the with block.

Returning exits the with block, which closes f before the caller starts iterating the generator expression. Making lines itself a generator and yielding inside with keeps the file open while that generator is active. Consumers must still finish or explicitly close an abandoned generator to ensure timely cleanup.

Why the alternatives fail

A. Keeping a reference to a file object does not prevent with from closing it.

C. The file is already closed by the context manager; closing earlier makes the same defect explicit.

D. Changing the cursor cannot make an already closed file readable.

Design takeaway: Keep resource lifetime aligned with the lifetime of lazy work.

Official references: [8], [4]

11. Nullable does not mean omittable

Answer A: Result() fails, while Result(reason=None) succeeds; the declaration matches the contract.

The annotation permits two kinds of value: a string or None. It does not provide a default. Therefore omitting reason is a missing-field error, while explicitly supplying None is valid. Adding = None would change the contract by allowing clients to omit the key.

Why the alternatives fail

B. In Pydantic 2, the nullable annotation alone does not supply a default.

C. A union containing None is supported and represents a nullable field.

D. A supplied None is a value allowed by the annotation, not an absent key.

Design takeaway: Specify presence and nullability independently.

Official references: [10]

12. A retry count must reject coercion

Answer C: Only 3 succeeds.

StrictInt accepts the integer 3 while rejecting the string and boolean. Python considers bool a subclass of int, but Pydantic's StrictInt deliberately rejects bool. This is useful when silently interpreting true as one retry would conceal a caller error.

Why the alternatives fail

A. StrictInt rejects bool despite Python's bool/int subclass relationship.

B. StrictInt disables the numeric-string coercion a normal int field may perform.

D. Both the numeric string and the boolean violate this strict contract.

Design takeaway: Choose coercion deliberately rather than relying on a type name alone.

Official references: [11]

13. A bad default bypasses ordinary field validation

Answer C: Add validate_default=True to Field(...).

Pydantic does not validate defaults by default. validate_default=True makes the default pass through validation, so -1 violates gt=0 and raises a ValidationError. Strictness controls accepted input types; it does not itself turn on default validation.

Why the alternatives fail

A. Relaxing the constraint neither enables default validation nor makes -1 valid.

B. An integer -1 is already an integer, and strictness alone does not validate the default.

D. model_construct bypasses validation and cannot enforce this constraint.

Design takeaway: Configuration defaults need validation just as externally supplied values do.

Official references: [10]

14. A misspelled setting silently disappears

Answer B: Set model_config = ConfigDict(extra="forbid").

The default extra-field behavior ignores unknown input keys, so the typo is dropped and the real field uses its default. extra="forbid" turns unknown keys into validation errors. This catches configuration mistakes at ingestion rather than allowing unexpected retry behavior later.

Why the alternatives fail

A. Allowing extras stores the misspelling without making it the actual retry setting.

C. Strictness concerns accepted value types; it does not by itself reject extra keys.

D. Nullability changes the accepted value domain, not unknown-key handling.

Design takeaway: Define an explicit policy for unknown fields at every external contract.

Official references: [12]

15. Normalize shape before validating items

Answer A: [7]; the mixed list fails integer-item validation.

The before validator wraps a scalar in a list before the declared list[int] validation runs. The string 7 can then be parsed into an integer. The same item validation rejects bad; there is no code that removes failed items or turns validation into a best-effort filter.

Why the alternatives fail

B. A before validator returns input for the normal type-validation pipeline; it does not bypass that pipeline.

C. Pydantic reports an invalid item instead of silently dropping it.

D. Before validators run before the declared field type is parsed.

Design takeaway: Normalize representation first, then enforce the resulting data contract.

Official references: [13]

16. An invariant spans two parsed fields

Answer D: An after model validator compares self.end and self.start, raises ValueError on failure, and returns self.

An after model validator operates on the parsed model and can inspect both fields together. It can reject a reversed or zero-length window with a clear validation error. A field validator that reads other fields is sensitive to validation order, which conflicts with the stated requirement.

Why the alternatives fail

A. An end greater than zero can still be less than or equal to start.

B. Raw input and field-order dependence make this unsuitable for the requested invariant.

C. A serialization-time check lets invalid models enter business logic before output is produced.

Design takeaway: Validate relationships at the level that owns all participating values.

Official references: [13]

17. A JSON-ready mapping is not a JSON string

Answer D: payload.model_dump(mode="json")

JSON mode returns a dictionary with values converted to JSON-compatible representations, including strings for the datetime and Decimal here. model_dump_json instead returns serialized JSON text. Passing that string where the client expects a mapping can lead to double encoding or a wrong request shape.

Why the alternatives fail

A. The standard encoder cannot encode these datetime and Decimal objects without additional handling.

B. Python mode retains Python datetime and Decimal objects, which do not meet the client's requirement.

C. This returns a JSON string, not the requested dictionary.

Design takeaway: Keep Python objects, JSON-compatible values, and serialized JSON text distinct.

Official references: [14]

18. A patch must distinguish omitted from explicit null

Answer C: model_dump(exclude_unset=True): a gives {}, b gives {"note": None}.

exclude_unset uses which fields were explicitly supplied. a has no supplied fields, while b explicitly supplies note, even though its value equals the default. exclude_none would drop the intended clearing operation; exclude_defaults would also drop this value because it equals the default.

Why the alternatives fail

A. The ordinary dump includes note=None in both objects, making omission look like a clear.

B. exclude_none removes note from both objects, losing the explicit request to clear it.

D. exclude_defaults removes values equal to their defaults, including b's explicit None.

Design takeaway: Partial updates need presence information in addition to field values.

Official references: [14]

19. Input and output field names are separate decisions

Answer B: {"requestId": "r1"}

validation_alias selects the accepted incoming key for this field. serialization_alias selects its outgoing name when alias serialization is requested. Therefore the field is populated from request-id and emitted as requestId. These names can differ without introducing extra model fields.

Why the alternatives fail

A. The internal Python name is used by an ordinary dump, not this alias-enabled dump.

C. The validation alias does not control the serialized key when a serialization alias is supplied.

D. Aliases refer to one field; they do not create duplicate output fields.

Design takeaway: Treat external read and write contracts as explicit mappings.

Official references: [15]

20. Nested validation errors identify the failing element

Answer A: ('items', 1, 'qty')

The error location follows the structure: the outer items field, index 1 of its list, then the nested qty field. List indexes are zero-based. This machine-readable path lets an API or review UI point to the exact bad input without parsing a human-readable error message.

Why the alternatives fail

B. The second list item has index 1, not 2.

C. This loses the outer field and item index needed to identify the failing value.

D. The invalid value is separate error data; it is not a structural location component.

Design takeaway: Use structured error paths to make validation failures actionable.

Official references: [16]

Python-Pydantic-M-Set1

Version assumptions and official sources

Code behavior checked on Python 3.12.14 and Pydantic 2.14.1. Python questions assume ordinary GIL-enabled CPython where relevant. Documentation checked October 28, 2026. Standard imports are omitted from snippets.

The questions and scenarios are original. API behavior is supported by execution checks and the official references below; architecture answers apply those behaviors to the stated constraints. Review the explanation, then articulate why each alternative fails.

[1] Python: default argument values
https://docs.python.org/3.12/tutorial/controlflow.html#default-argument-values

[2] Python: shallow and deep copying
https://docs.python.org/3.12/library/copy.html

[3] Python: evaluation order and calls
https://docs.python.org/3.12/reference/expressions.html#evaluation-order

[4] Python: iterator types
https://docs.python.org/3.12/library/stdtypes.html#iterator-types

[5] Python FAQ: lambdas defined in loops
https://docs.python.org/3.12/faq/programming.html#why-do-lambdas-defined-in-a-loop-with-different-values-all-return-the-same-result

[6] Python: zip and strict length checking
https://docs.python.org/3.12/library/functions.html#zip

[7] Python: TypedDict runtime behavior
https://docs.python.org/3.12/library/typing.html#typing.TypedDict

[8] Python: try, finally, and with statements
https://docs.python.org/3.12/reference/compound_stmts.html#the-try-statement

[9] Python: exception chaining
https://docs.python.org/3.12/tutorial/errors.html#exception-chaining

[10] Pydantic: required fields, defaults, and field settings
https://docs.pydantic.dev/latest/concepts/fields/

Python-Pydantic-M-Set1

Version assumptions and official sources

Code behavior checked on Python 3.12.14 and Pydantic 2.14.1. Python questions assume ordinary GIL-enabled CPython where relevant. Documentation checked October 28, 2026. Standard imports are omitted from snippets.

The questions and scenarios are original. API behavior is supported by execution checks and the official references below; architecture answers apply those behaviors to the stated constraints. Review the explanation, then articulate why each alternative fails.

[11] Pydantic: strict types
https://docs.pydantic.dev/latest/api/types/#pydantic.types.StrictInt

[12] Pydantic: configuration API
https://docs.pydantic.dev/latest/api/config/

[13] Pydantic: field and model validators
https://docs.pydantic.dev/latest/concepts/validators/

[14] Pydantic: serialization modes and field selection
https://docs.pydantic.dev/latest/concepts/serialization/

[15] Pydantic: validation and serialization aliases
https://docs.pydantic.dev/latest/concepts/alias/

[16] Pydantic: structured validation errors
https://docs.pydantic.dev/latest/errors/errors/